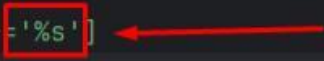


Locator handling for TextFieldElement:

1 Locator Template (properties file)

- **Description:** A raw XPath template with %s as a placeholder.
- **Purpose:** One template → reusable for multiple fields. Keeps locators DRY and easy to maintain.
- **OOP Principle: Abstraction** — hides locator complexity behind a simple template.

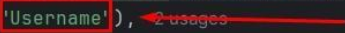
```
LOGIN_INPUT = //input[@data-placeholder='%s']
```



2 Enum Field Definition

- **Description:** Enum constants represent UI fields, each storing a label.
- **Purpose:** Strong typing + IDE autocomplete. Labels double as arguments for templates.
- **OOP Principle: Encapsulation** — field definitions + labels are bundled together.

```
// Text Fields
public enum LoginTextFieldElement implements TextFieldElement, ResolvableEnum { 8 usages
    USERNAME( label: 'Username'), 2 usages
    EMAIL( label: "Email"), 3 usages
```



3 Enum Arguments via getArgs()

- **Description:** Enum exposes its label as an argument.
- **Purpose:** Clean injection of field names into locator templates. Flexible resolution.
- **OOP Principle: Polymorphism** — every enum can expose args differently if needed.

```
public Object[] getArgs() { return new Object[]{label}; }
```



4 LocatorReader (Resolver)

- **Description:** Reads templates, fetches args, applies them → final XPath.
- **Purpose:** Decouples locators from code, ensures consistent resolution.
- **OOP Principle: Abstraction + Encapsulation** — resolution logic is hidden and reusable.

```

Enters text into a TextFieldElement.

Params: element - The TextFieldElement enum representing the field.
        text - The text to enter.

Throws: RuntimeException - if input fails.

public void inputText(TextFieldElement element, String text) {
    try {
        By locator = LocatorReader.getLocator(element);

```

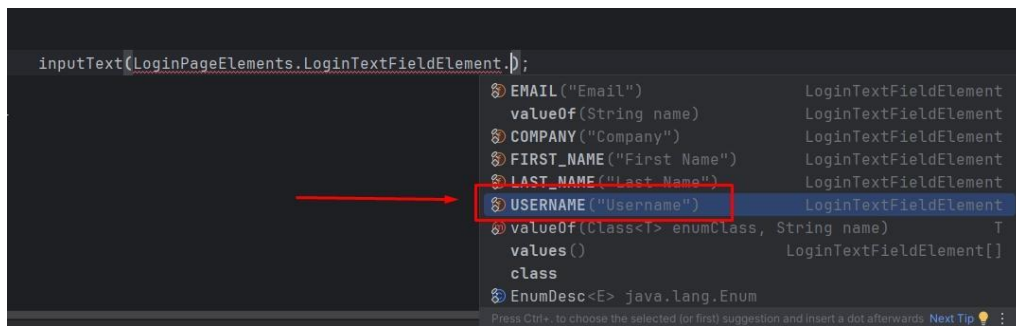
5 Clean Test Call

- **Description:** The code you actually write in tests.
- **Purpose:** Readable, no raw XPath — framework handles plumbing.
- **OOP Principle: Inheritance** — test code benefits from reusable parent logic (framework).

```

inputText(LoginPageElements.LoginTextFieldElement.);

```



```

inputText(LoginTextFieldElement.USERNAME, text: "username@xyz.com");

```

6 Runtime Logs (End-to-End)

- **Description:** Transparent trace of what happened.
- **Purpose:** Debugging made easy — file, key, args, resolved XPath, and action result all shown.
- **OOP Principle: Encapsulation** — internals are visible in logs only, not mixed into test code.

```

[DEBUG] LoginPageUsersInteractions.filterBy putting 'username@xyz.com' into field: Username
[INPUT] LoginPageUsersInteractions.filterBy Entering 'username@xyz.com' in field: Username
[DEBUG] LocatorReader.getLocator [LOCATOR] Resolving locator:
[DEBUG] LocatorReader.getLocator | Property File: login-page-elements.properties
[DEBUG] LocatorReader.getLocator | Key : LOGIN_INPUT
[DEBUG] LocatorReader.getLocator | Args : [Username]
[DEBUG] LocatorReader.getLocator [LOCATOR] Raw locator template:
[DEBUG] LocatorReader.getLocator | Template : //input[@data-placeholder='%s']
[DEBUG] LocatorReader.getLocator [LOCATOR] Final resolved XPath:
[DEBUG] LocatorReader.getLocator | Key : LOGIN_INPUT
[DEBUG] LocatorReader.getLocator | Final XPath : //input[@data-placeholder='Username']
[INPUT] Interactions.inputText Entered username@xyz.com into Username
[SUCCESS] Interactions.inputText Entered username@xyz.com into Username

```

🌟 TL;DR:

- **Abstraction** → Templates hide complexity.
 - **Encapsulation** → Enums bundle field + label.
 - **Polymorphism** → Flexible `getArgs()` handling.
 - **Inheritance** → Tests stay clean, extend framework behavior.
-